AVL Tree with RL Rotation

CCE-2 · Unit 4 · C++ Lab Report

1. Title

Insertion in an AVL Tree with RL (Right-Left) Rotation

2. Problem Statement

Write a C++ program to insert the elements 10, 30 and 20 into an AVL tree and perform the required RL rotation to restore balance.

3. Objectives

- Understand what an AVL tree is (a self-balancing binary search tree).
- Understand balance factor = height(left) − height(right). It must be −1, 0 or +1.
- Learn when the RL case happens: the new node goes into the left subtree of the right child.
- Perform the double rotation (right rotation, then left rotation) step by step.
- Print balance factors before and after rebalancing.
- Test a second sequence that also gives an RL imbalance.

4. Concept: The RL Case

A node becomes unbalanced when its balance factor falls outside −1 to +1. The RL case occurs when both of these hold:

Condition	Meaning
Balance factor of the node is less than −1	The right side is too tall (right-heavy)
New key is smaller than the right child's key	The new node went into the right child's left subtree

A single left rotation cannot fix this, so two rotations are needed: first a right rotation on the right child, then a left rotation on the unbalanced node.

Worked example (insert 10, 30, 20):

1. Insert 10 and 30. The tree is balanced (10 has balance factor −1).
2. Insert 20. It becomes the left child of 30. Node 10 now has balance factor −2 and its right child 30 has +1, so this is an RL case.
3. Step 1: right rotate on 30. The tree becomes a right chain 10 → 20 → 30.
4. Step 2: left rotate on 10. Node 20 becomes the root with 10 on the left and 30 on the right. Every balance factor is 0.

5. Algorithm

Step 1: Insert a key like in a normal BST.

Step 2: On the way back up, update each node's height.

Step 3: Calculate the balance factor of each node.

Step 4: If the balance factor is between −1 and +1, do nothing.

Step 5: If the balance factor is less than −1 and the new key is smaller than the right child's key, it is the RL case.

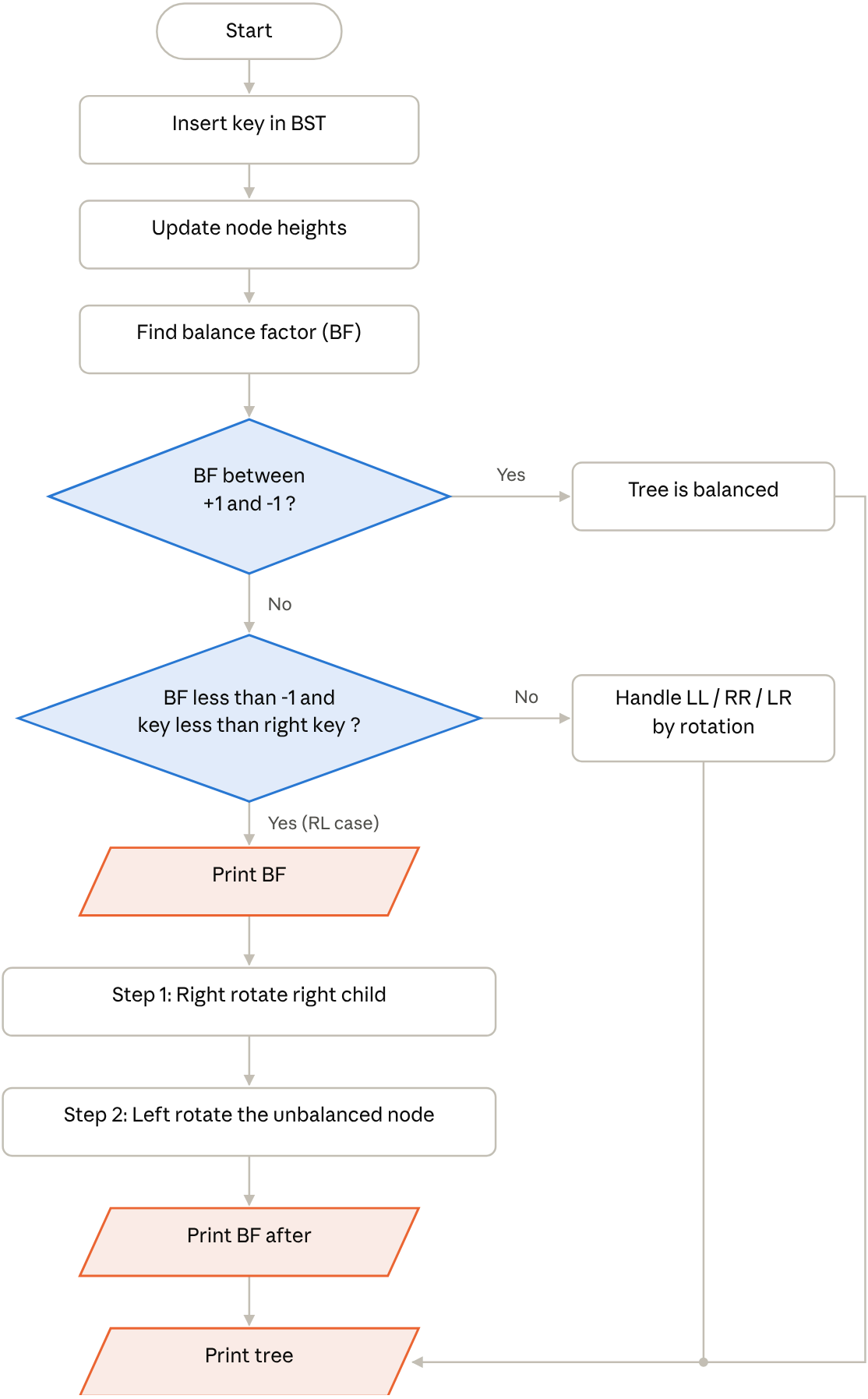
- Step 5.1: Right-rotate the right child.
- Step 5.2: Left-rotate the unbalanced node.
- Print the balance factors after.

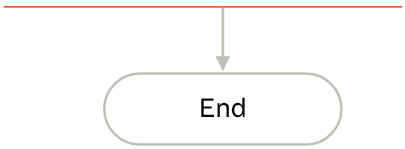
Step 6: (The LL, RR and LR cases are handled in the same way, for completeness.)

Step 7: Print the tree.

6. Flowchart

(See flowchart below.)


Start
Insert key in BST
Update node heights
Find balance factor (BF)
BF between +1 and -1 ?
Yes
Tree is balanced
No
BF less than -1 and key less than right key ?
No
Handle LL / RR / LR by rotation
Yes (RL case)
Print BF
Step 1: Right rotate right child
Step 2: Left rotate the unbalanced node
Print BF after
Print tree



AVL insertion flow · RL case = right rotate, then left rotate

Only the RL case (BF below -1 and new key smaller than the right child's key) follows the two-step rotation; other cases are handled by their own rotations.

7. C++ Program

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

struct Node {
    int key, height;
    Node *left, *right;
    Node(int k) : key(k), height(1), left(NULL), right(NULL) {}
};

int height(Node* n) { return n ? n->height : 0; }

int balance(Node* n) { return n ? height(n->left) - height(n->right) : 0; }

void update(Node* n) { n->height = 1 + max(height(n->left), height(n-
>right)); }

// Show the tree sideways (right side on top) with balance factors
void show(Node* n, int space = 0) {
    if (!n) return;
    show(n->right, space + 6);
    cout << string(space, ' ') << n->key << "(bf=" << balance(n) << ")\n";
    show(n->left, space + 6);
}

Node* rotateRight(Node* y) {
    Node* x = y->left;
    y->left = x->right;
    x->right = y;
    update(y);
```

```
    update(x);
    return x;
}

Node* rotateLeft(Node* x) {
    Node* y = x->right;
    x->right = y->left;
    y->left = x;
    update(x);
    update(y);
    return y;
}

Node* insert(Node* node, int key) {
    // Normal BST insert
    if (!node) return new Node(key);
    if (key < node->key)      node->left  = insert(node->left, key);
    else if (key > node->key) node->right = insert(node->right, key);
    else return node;                       // no duplicates

    update(node);
    int bf = balance(node);

    // LL case
    if (bf > 1 && key < node->left->key)
        return rotateRight(node);

    // RR case
    if (bf < -1 && key > node->right->key)
        return rotateLeft(node);

    // LR case
    if (bf > 1 && key > node->left->key) {
        node->left = rotateLeft(node->left);
        return rotateRight(node);
    }

    // RL case
    if (bf < -1 && key < node->right->key) {
        cout << "\n*** RL imbalance found at node " << node->key << " ***\n";

        cout << "Balance factors BEFORE rebalancing:\n";
        cout << "  Node " << node->key << "        : " << balance(node) <<
```

```
"\n";
        cout << "  Right child " << node->right->key << " : "
             << balance(node->right) << "\n";

        cout << "\nStep 1: Right rotate on " << node->right->key << "\n";
        node->right = rotateRight(node->right);
        show(node);

        cout << "\nStep 2: Left rotate on " << node->key << "\n";
        Node* newRoot = rotateLeft(node);
        show(newRoot);

        cout << "\nBalance factors AFTER rebalancing:\n";
        cout << "  New root " << newRoot->key << "  : " << balance(newRoot)
<< "\n";
        cout << "  Left  " << newRoot->left->key << "      : "
             << balance(newRoot->left) << "\n";
        cout << "  Right " << newRoot->right->key << "      : "
             << balance(newRoot->right) << "\n";
        return newRoot;
    }
    return node;
}

void test(int arr[], int n, const char* title) {
    cout << "\n==============================\n";
    cout << title << "\n";
    cout << "==============================\n";
    Node* root = NULL;
    for (int i = 0; i < n; i++) {
        cout << "\nInserting " << arr[i] << "...\n";
        root = insert(root, arr[i]);
    }
    cout << "\nFinal AVL tree:\n";
    show(root);
}

int main() {
    int seq1[] = {10, 30, 20};
    test(seq1, 3, "Test 1: 10, 30, 20");

    // Second RL test: 10 becomes -2, its right child 30 is left-heavy
    int seq2[] = {10, 5, 30, 20, 40, 25};
```

```
    test(seq2, 6, "Test 2: 10, 5, 30, 20, 40, 25");

    return 0;
}
```

8. Sample Output

The tree is printed sideways: the right subtree is on top, the root is at the left margin, and the left subtree is below.

Test 1: 10, 30, 20

```
Inserting 10...

Inserting 30...

Inserting 20...

*** RL imbalance found at node 10 ***
Balance factors BEFORE rebalancing:
  Node 10        : -2
  Right child 30 : 1

Step 1: Right rotate on 30
            30(bf=0)
      20(bf=-1)
10(bf=-2)

Step 2: Left rotate on 10
      30(bf=0)
20(bf=0)
      10(bf=0)

Balance factors AFTER rebalancing:
  New root 20  : 0
  Left  10     : 0
  Right 30     : 0

Final AVL tree:
      30(bf=0)
20(bf=0)
      10(bf=0)
```

Test 2: 10, 5, 30, 20, 40, 25

```
*** RL imbalance found at node 10 ***
Balance factors BEFORE rebalancing:
  Node 10        : -2
  Right child 30 : 1

Step 1: Right rotate on 30
                  40(bf=0)
            30(bf=0)
                  25(bf=0)
      20(bf=-2)
10(bf=-2)
      5(bf=0)

Step 2: Left rotate on 10
            40(bf=0)
      30(bf=0)
            25(bf=0)
20(bf=0)
      10(bf=1)
            5(bf=0)

Balance factors AFTER rebalancing:
  New root 20  : 0
  Left  10     : 1
  Right 30     : 0

Final AVL tree:
            40(bf=0)
      30(bf=0)
            25(bf=0)
20(bf=0)
      10(bf=1)
            5(bf=0)
```

9. Time Complexity

Operation	Complexity	Reason
Insert	O(log n)	AVL height stays about log n, plus O(1) work per node on the way back up
Single or double rotation	O(1)	Only a few pointers change
Search	O(log n)	Tree height is kept small by balancing
Space	O(n)	One node per key (recursion adds O(log n))

10. Conclusion

The program inserted the elements into an AVL tree, detected the RL imbalance through the balance factor, and restored balance with a right rotation on the right child followed by a left rotation on the unbalanced node. Printing the balance factors before and after showed the tree moving from −2 back to values within −1 to +1. A second test sequence confirmed that the same double rotation works on a larger tree.